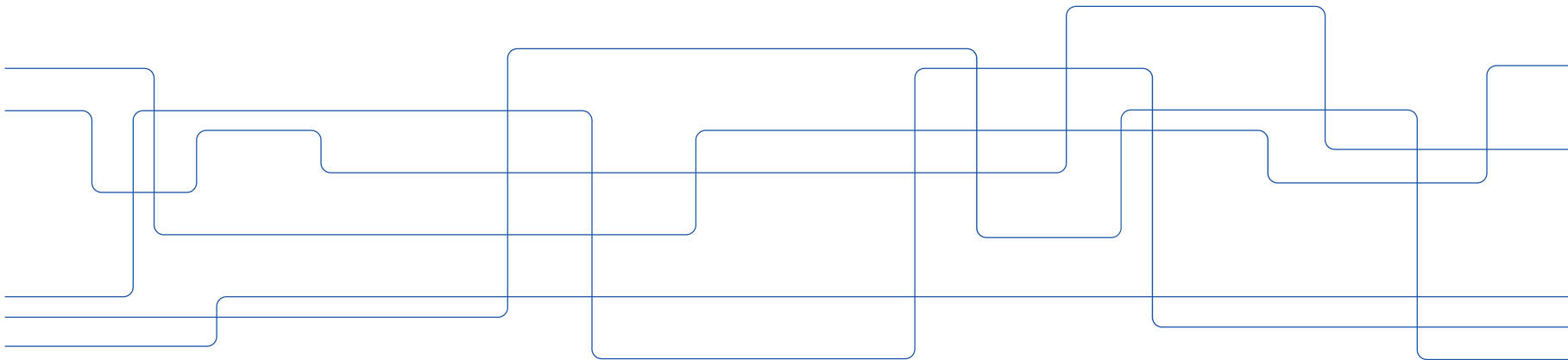


Logistic Regression as a simple Neural Network (Build the general architecture of a learning algorithm, (gradient descent))





What we learn at the end of this session

- Recap of notations, logistic regression cost function and gradient descent
- Computation graph
- Implementing computation graph on logistic regression
- Logistic regression algorithm
- Vectorization in Logistic regression
- Implementing Logistic Regression

Notation (example)

$$X = \begin{bmatrix} x_1^{(1)} & \dots & x_1^{(200)} \\ x_2^{(1)} & \dots & x_2^{(200)} \end{bmatrix} \begin{matrix} \uparrow \\ n_x = 2 \\ \downarrow \end{matrix}$$

$$\longleftarrow m = 200 \longrightarrow$$

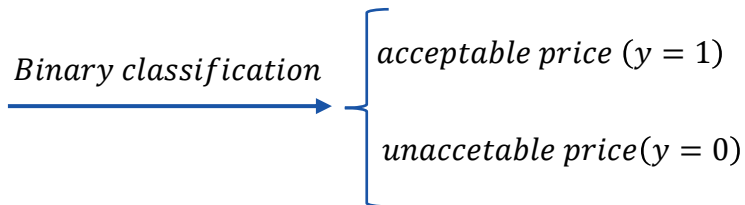
$$X \in \mathbb{R}^{2 \times 200} \quad X.\text{shape} = (2, 200)$$

$$Y = [1, 1, 0, \dots, 0]$$

$$Y \in \mathbb{R}^{1 \times 200}$$

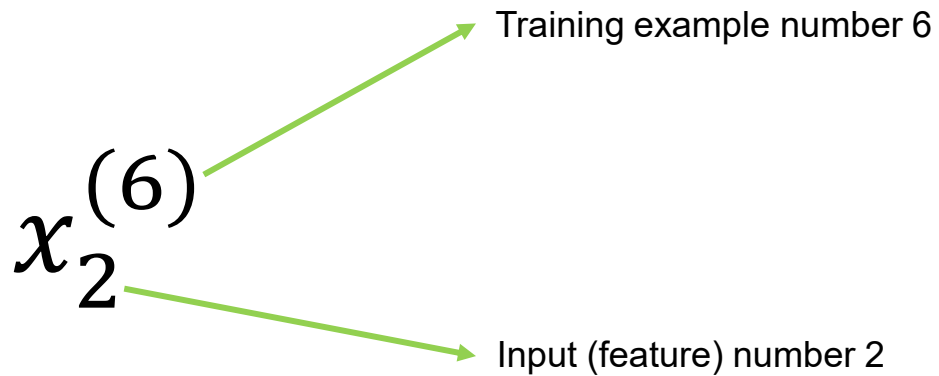
$$Y.\text{shape} = (1, 200)$$

Training example	X1=Efficiency (%)	X2=Material	Price (\$/W)	Y=acceptable price
1	15	silicon	0.3	1
2	20	silicon	0.5	1
3	22	Monocrystalline	1.0	0
...	...	...	...	...
200	18	Polycrystalline	0.7	0





Notation





What Is Logistic Regression?

1. Start like Linear Regression

Suppose we want to predict whether a machine will **fail (1)** or **not fail (0)** based on temperature x .

We first calculate a linear score:

$$z = b + w_1x$$

For multiple inputs:

$$z = b + w_1x_1 + w_2x_2 + \cdots + w_nx_n$$

For example:

$$z = -5 + 0.1 \times \textit{Temperature}$$

If temperature = 62°C:

$$z = -5 + 0.1(62) = 1.2$$

The problem is that z can be anything:

$$-\infty < z < +\infty$$

But we want a **probability between 0 and 1**.

What Is Logistic Regression?

2. Pass it through the Sigmoid function

Logistic Regression uses the **sigmoid (logistic) function**:

$$P = \frac{1}{1 + e^{-z}}$$

This converts any number into something between **0** and **1**.

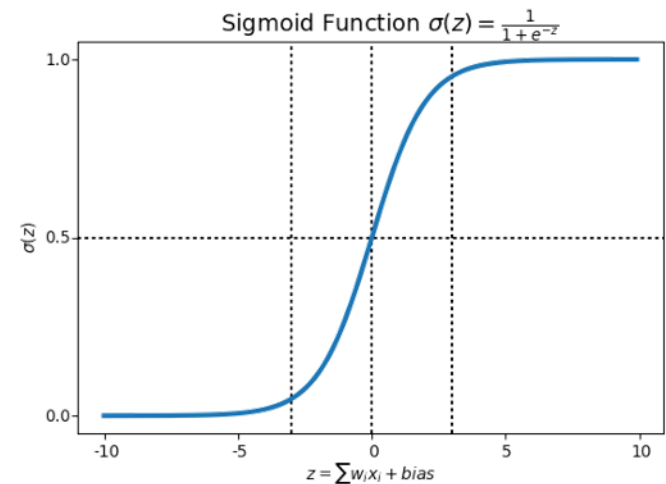
For our example, $z = 1.2$:

$$P = \frac{1}{1 + e^{-1}} \approx 0.76$$

So the model says:

Probability of machine failure = 76%

The sigmoid has an S-shape:





What Is Logistic Regression?

3. Convert probability into a class

We then choose a threshold, often **0.5**:

$$P > 0.5 \rightarrow \textit{Class 1}$$

$$P \leq 0.5 \rightarrow \textit{Class 0}$$

So if:

$$P = 0.73 \rightarrow \textbf{Failure}$$

and:

$$P = 0.20 \rightarrow \textbf{No failure}$$

But how does the model learn $b, w_1, \dots$?

During training, Logistic Regression searches for coefficients that make the observed classes as probable as possible.

For example, suppose the real result was **Failure = 1**:

Prediction 0.95 $\rightarrow$ very good

Prediction 0.80 $\rightarrow$ good

Prediction 0.10 $\rightarrow$ very bad

It measures these errors using a **cross-entropy loss function**, and an optimization algorithm adjusts the coefficients to minimize that loss.

Logistic Regression

Given X , we want $\hat{y} = P(y = 1|X)$

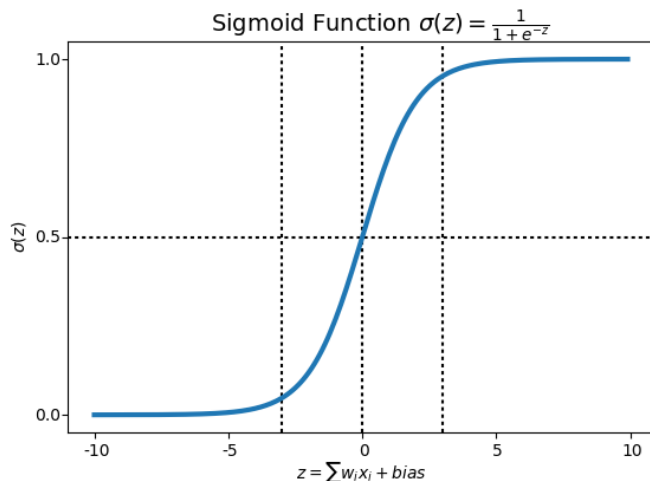
$X \in \mathbb{R}^{n_x}$

Parameters : $\mathbf{w} \in \mathbb{R}^{n_x}$ and $\mathbf{b} \in \mathbb{R}$

Output: $\hat{y} = \sigma(\underbrace{\mathbf{w}^T X + \mathbf{b}}_z)$

$$0 \leq \hat{y} \leq 1$$

So when you implement logistic regression, your job is to try to learn parameters $\mathbf{w}$ and $\mathbf{b}$ so that $\hat{y}$ becomes a good estimate of the chance of y being equal to 1.



$$\sigma(z) = \frac{1}{1+e^{-z}}$$

If z large, $\sigma(z) \approx \frac{1}{1+0} = 1$
 If z large negative number,

$$\sigma(z) = \frac{1}{1+\text{big number}} \approx 0$$



Logistic Regression Process

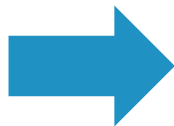
X				Y
X1	X2	...	Xn	Y
2	48	...	10	1
4	75	...	8	0
6	20	...	24	0
...	...	...	...	...
3	45	...	9	1

Data set

Logistic Regression Process

X				Y
X1	X2	...	Xn	Y
2	48	...	10	1
4	75	...	8	0
6	20	...	24	0
...	...	...	...	...
3	45	...	9	1

Data set



$$\hat{y} = \sigma(w^T X + b)$$



Initial guess
for W and b

Logistic Regression Process

X				Y
X1	X2	...	Xn	Y
2	48	...	10	1
4	75	...	8	0
6	20	...	24	0
...	...	...	...	...
3	45	...	9	1

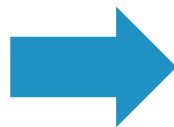
Data set



$$\hat{y} = \sigma(w^T X + b)$$



Initial guess
for W and b



X1	X2	...	Xn	Y	Y_hat
2	48	...	10	1	0
4	75	...	8	0	1
6	20	...	24	0	0
...	...	...	...	...	...
3	45	...	9	1	1

Logistic Regression Process

X				Y
X1	X2	...	Xn	Y
2	48	...	10	1
4	75	...	8	0
6	20	...	24	0
...	...	...	...	...
3	45	...	9	1

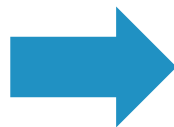
Data set



$$\hat{y} = \sigma(w^T X + b)$$



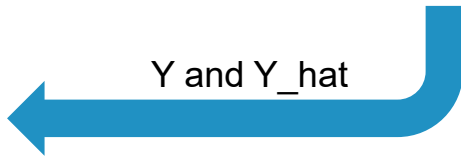
Initial guess
for W and b



X1	X2	...	Xn	Y	Y_hat
2	48	...	10	1	0
4	75	...	8	0	1
6	20	...	24	0	0
...	...	...	...	...	...
3	45	...	9	1	1

Y and Y_hat

Error



Logistic Regression Process

X				Y
X1	X2	...	Xn	Y
2	48	...	10	1
4	75	...	8	0
6	20	...	24	0
...	...	...	...	...
3	45	...	9	1

Data set



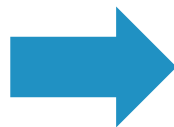
$$\hat{y} = \sigma(w^T X + b)$$



Initial guess
for W and b



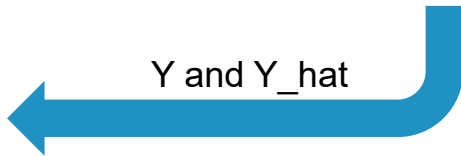
Update W and b



X1	X2	...	Xn	Y	Y_hat
2	48	...	10	1	0
4	75	...	8	0	1
6	20	...	24	0	0
...	...	...	...	...	...
3	45	...	9	1	1

Y and Y_hat

Error



Logistic Regression Process

X				Y
X1	X2	...	Xn	Y
2	48	...	10	1
4	75	...	8	0
6	20	...	24	0
...	...	...	...	...
3	45	...	9	1

Data set

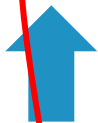


$$\hat{y} = \sigma(w^T X + b)$$



X1	X2	...	Xn	Y	Y_hat
2	48	...	10	1	0
4	75	...	8	0	1
6	20	...	24	0	0
...	...	...	...	...	...
3	45	...	9	1	1

Initial guess
for W and b



Update W and b

Error

Optimal W and b

Min Error

Y and Y_hat

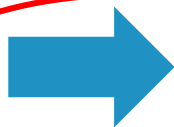
Logistic Regression Process

X				Y
X1	X2	...	Xn	Y
2	48	...	10	1
4	75	...	8	0
6	20	...	24	0
...	...	...	...	...
3	45	...	9	1

Data set



$$\hat{y} = \sigma(w^T X + b)$$



X1	X2	...	Xn	Y	Y_hat
2	48	...	10	1	0
4	75	...	8	0	1
6	20	...	24	0	0
...	...	...	...	...	...
3	45	...	9	1	1

Initial guess
for W and b



Update W and b

?

Error

Y and Y_hat

Optimal W and b

Min Error

Logistic Regression Process

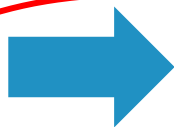
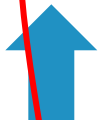
X				Y
X1	X2	...	Xn	Y
2	48	...	10	1
4	75	...	8	0
6	20	...	24	0
...	...	...	...	...
3	45	...	9	1

Data set



$$\hat{y} = \sigma(w^T X + b)$$

Initial guess
for W and b



X1	X2	...	Xn	Y	Y_hat
2	48	...	10	1	0
4	75	...	8	0	1
6	20	...	24	0	0
...	...	...	...	...	...
3	45	...	9	1	1

Cost function

=

Error

Y and Y_hat

Update W and b

Optimal W and b

Min Error



Logistic Regression cost function

Loss function: apply on a single training example

$$L(\hat{y}, y) = -(y \log \hat{y} + (1 - y) \log(1 - \hat{y}))$$

Cost function: apply on a overall training example

$$\begin{aligned} J(W, b) \\ = \frac{1}{m} \sum_{i=1}^m L(\hat{y}^{(i)}, y^{(i)}) &= -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})] \end{aligned}$$

Logistic Regression cost function

Loss function: apply on a single training example

$$L(\hat{y}, y) = -(y \log \hat{y} + (1 - y) \log(1 - \hat{y}))$$

Cost function: apply on a overall training example

$$\begin{aligned} J(W, b) \\ = \frac{1}{m} \sum_{i=1}^m L(\hat{y}^{(i)}, y^{(i)}) &= -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})] \end{aligned}$$

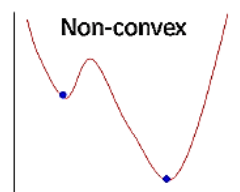
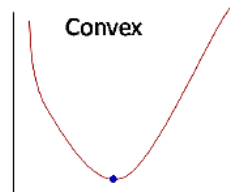
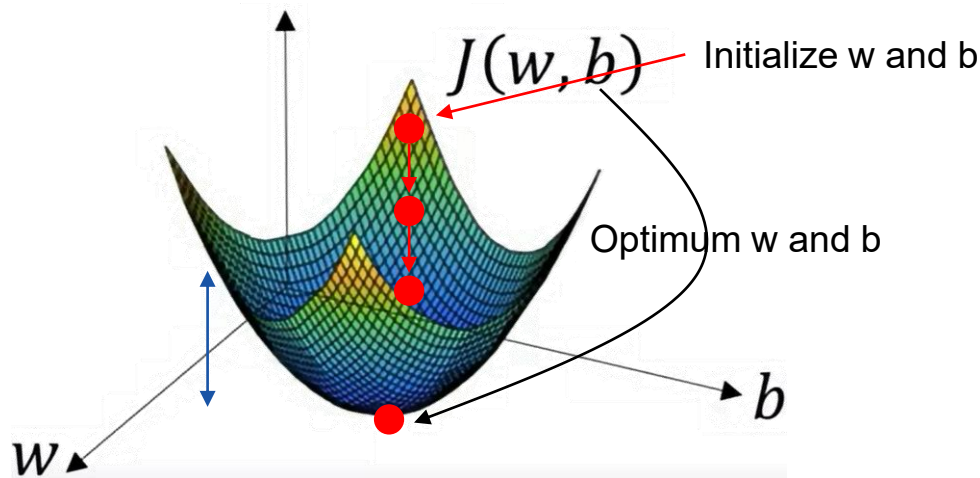
How to minimize the cost function?

Gradient Descent

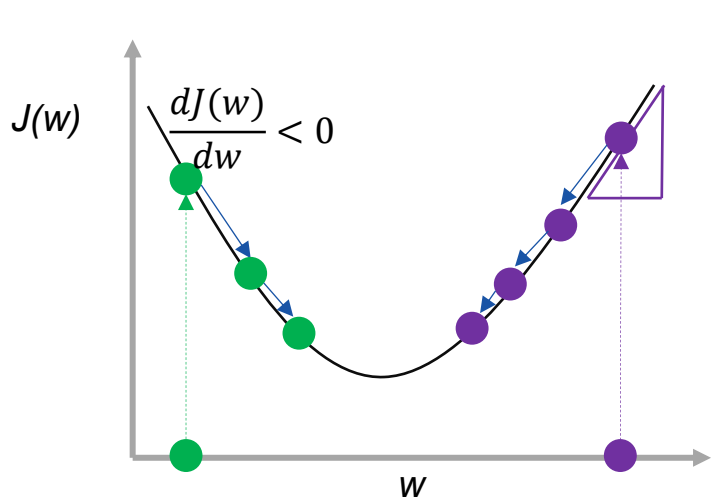
Want to find w, b that minimize $J(w, b)$

$$\hat{y} = \sigma(W^T X + b), \quad \text{where } \sigma(z) = \frac{1}{1 + e^{-z}}$$

$$J(W, b) = \frac{1}{m} \sum_{i=1}^m L(\hat{y}^{(i)}, y^{(i)}) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} - \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})]$$



Gradient Descent



Repeat {

Learning rate

$$w := w - \alpha \underbrace{\frac{dJ(w)}{dw}}_{\text{"dw"}}$$

}

In codes: $w = w - \alpha dw$

$$J(w, b) \quad w = w - \alpha \frac{dJ(w, b)}{dw}$$

$$b = b - \alpha \frac{dJ(w, b)}{db}$$

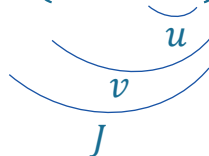
Partial derivative
Functions of more than 1 parameters

dw

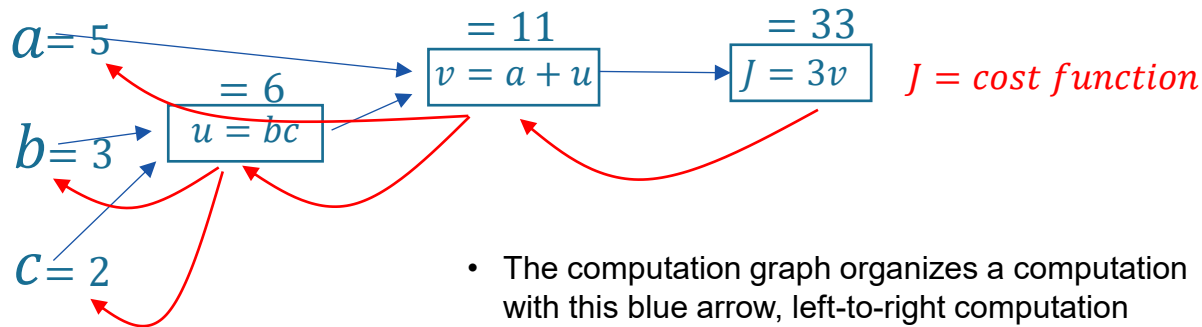
db

Computation graph (general example)

$$J(a, b, c) = 3(a + bc) = 3(5 + 3 \times 2) = 33$$

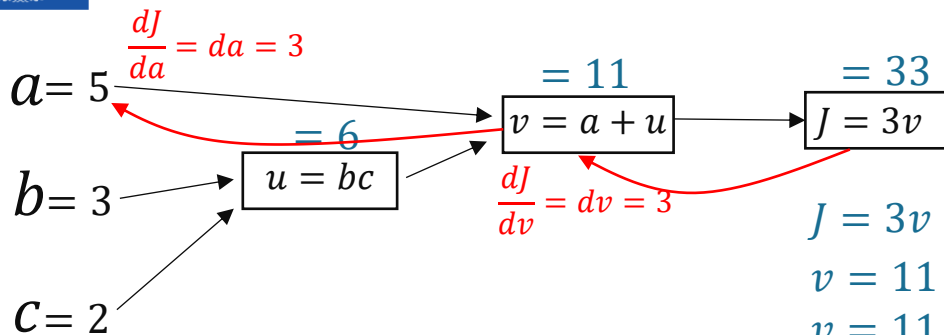


$$\begin{aligned} u &= bc \\ v &= a + u \\ J &= 3v \end{aligned}$$



- The computation graph organizes a computation with this blue arrow, left-to-right computation
- The backward red arrow right-to-left computation of the derivatives.

Computation Derivatives



$$J = 3v$$

$$v = 11 \rightarrow J = 33$$

$$v = 11.001 \rightarrow J = 33.003$$

$$f(a) = 3a$$

$$\frac{df(a)}{da} = 3$$

$$\frac{dJ}{dv} = 3$$

$$\frac{dJ}{dv} = ? = 3$$

$$\frac{dJ}{da} = ? = 3 = \frac{dJ}{dv} \times \frac{dv}{da}$$

$$\frac{dv}{da} = 1$$

$$a = 5 \rightarrow v = 11$$

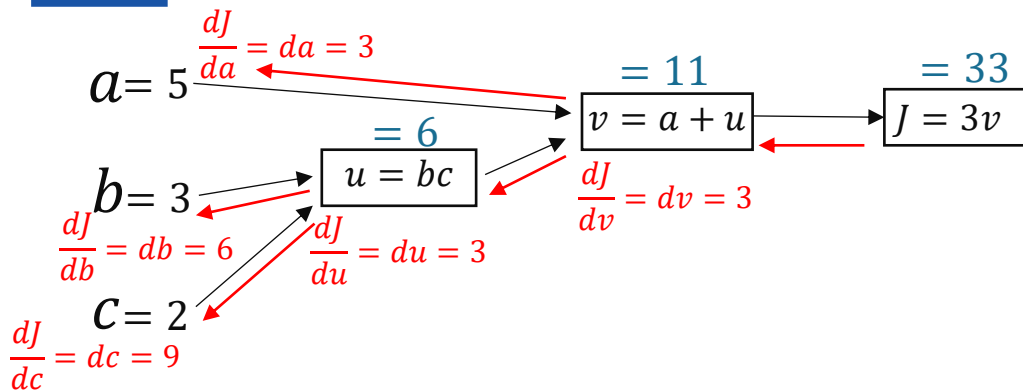
$$a = 5.001 \rightarrow v = 11.001$$

$$\rightarrow J = 33$$

$$\rightarrow J = 33.003$$

$$\frac{dFinalOutputVar}{dvar} = dvar$$

Computation Derivatives



$$\frac{dJ}{du} = ? = \frac{dJ}{dv} \times \frac{dv}{du} = 3$$

$u = 6$	$\rightarrow v = 11$	$\rightarrow J = 33$
$u = 6.001$	$\rightarrow v = 11.001$	$\rightarrow J = 33.003$

$$\frac{dJ}{db} = ? = \frac{dJ}{du} \times \frac{du}{db} = 6$$

$b = 3$	$\rightarrow u = 6$
$b = 3.001$	$\rightarrow u = 6.002$

$$\frac{dJ}{dc} = \frac{dJ}{du} \times \frac{du}{dc} = 9$$



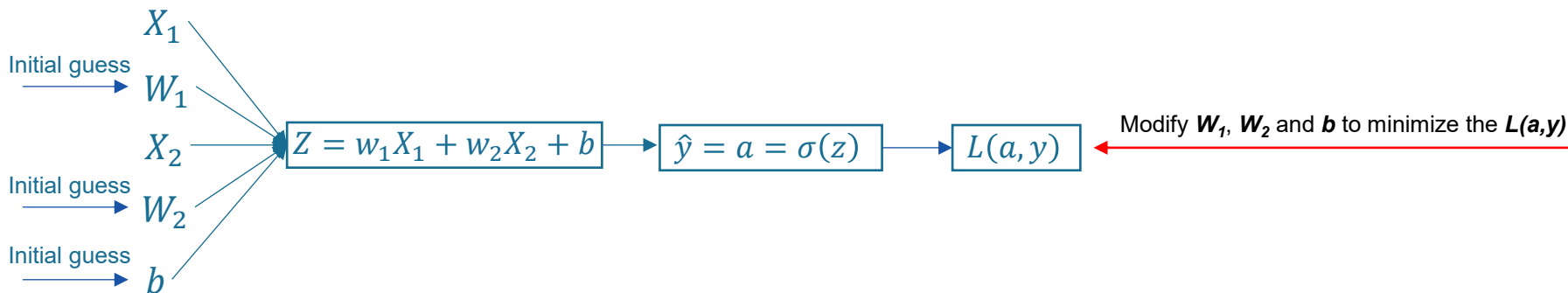
Logistic regression recap

$$z = w^T X + b$$

$$\hat{y} = a = \sigma(z)$$

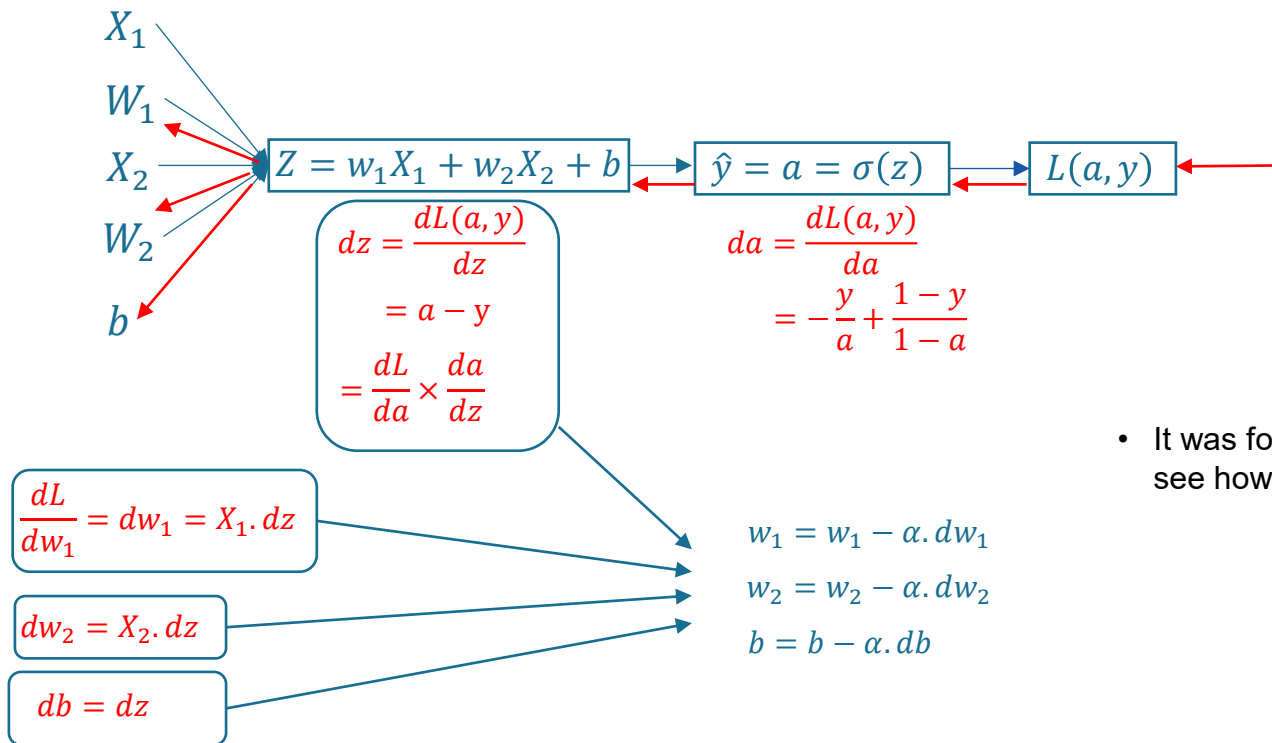
$$L(a, y) = -(y \log(a) + (1 - y) \log(1 - a))$$

One example with 2 input variables as X_1 and X_2



Logistic regression recap

$$\frac{da}{dz} = a(1 - a)$$



- It was for a single example. Now lets see how it is done for m examples



Logistic regression on m example

$$J(w, b) = \frac{1}{m} \sum_{i=1}^m L(a^{(i)}, y^{(i)})$$

$$a^{(i)} = \hat{y}^{(i)} = \sigma(z^{(i)}) = \sigma(w^T \cdot X^{(i)} + b)$$

In previous slide we learned dw_1 , dw_2 , and db just for 1 training example.

$$\frac{\partial}{\partial w_1} J(w, b) = \frac{1}{m} \sum_{i=1}^m \underbrace{\frac{\partial}{\partial w_1} L(a^{(i)} - y^{(i)})}_{dw_1^{(i)}: \text{for 1 training example}}$$

Lets put everything into a concrete algorithm

Logistic regression algorithm

Initiate

$$J = 0, dw_1 = 0, dw_2 = 0, db = 0$$

First for loop

for $i = 1$ to m

$$z^{(i)} = w^T X^{(i)} + b$$

$$a^{(i)} = \sigma(z^{(i)})$$

$$J += -[y^{(i)} \log a^{(i)} + (1 - y^{(i)}) \log(1 - a^{(i)})]$$

$$dz^{(i)} = a^{(i)} - y^{(i)}$$

$$dw_1 += x_1^{(i)} dz^{(i)}$$

$$dw_2 += x_2^{(i)} dz^{(i)}$$

$$db += dz^{(i)}$$

$n = 2$

$$J /= m$$

$$dw_1 /= m$$

$$dw_2 /= m$$

$$db /= m$$

No superscript
because of
accumulation

$$w_1 = w_1 - \alpha \cdot dw_1$$

$$w_2 = w_2 - \alpha \cdot dw_2$$

$$b = b - \alpha \cdot db$$

Second for loop

- Everything on this slide is just one **single step** in the gradient descent.
- You have to implement this slide, **multiple times**.



Vectorization

$$Z = w^T X + b$$

$$w = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_{n_x} \end{bmatrix} \quad X = \begin{bmatrix} X_1 \\ X_2 \\ \vdots \\ X_{n_x} \end{bmatrix}$$

Non-vectorized:

$$Z = 0$$

for i in range (n_x):

$$z += w[i] \times x[i]$$

$$z += b$$

Vectorized:

$$z = \underbrace{np.dot(w, X)}_{w^T X} + b$$

```
import time

w = np.random.rand(1000000)
X = np.random.rand(1000000)

tic = time.time()
z = np.dot(w, X)
toc = time.time()

print(z)
print("Vectorized version: " + str(1000*(toc-tic)) + "ms")

z = 0
tic = time.time()
for i in range(1000000):
    z += w[i]*X[i]
toc = time.time()

print(z)
print("For loop: " + str(1000*(toc-tic)) + "ms")
```

✓ 0.2s

250175.6952210248
Vectorized version:0.8041858673095703ms
250175.69522103193
For loop:234.71474647521973ms

The rule of thumb to remember is whenever possible, avoid using explicit for loops.

Vectorization in Logistic regression

$$J = 0, \text{ ~~dw = 0, db = 0~~ } db = 0$$

$$dw = np.zeros(n_x, 1)$$

for $i = 1$ to m

$$z^{(i)} = w^T X^{(i)} + b$$

$$a^{(i)} = \sigma(z^{(i)})$$

$$J += -[y^{(i)} \log a^{(i)} + (1 - y^{(i)}) \log(1 - a^{(i)})]$$

$$dz^{(i)} = a^{(i)} - y^{(i)}$$

Forward propagation
vectorization

for $j = 1$ to n_x
 $dw_j += x_j^{(i)} dz^{(i)}$

~~$$dw_1 += x_1^{(i)} dz^{(i)}$$

$$dw_2 += x_2^{(i)} dz^{(i)}$$~~

$$dw += X^{(i)} dz^{(i)}$$

$$db += dz^{(i)}$$

Back propagation
vectorization

$J /= m$

~~$$dw_1 /= m$$

$$dw_2 /= m$$~~

$$db /= m$$

$$dw /= m$$

Vectorization in forward propagation

$$z^{(1)} = w^T x^{(1)} + b$$

$$a^{(1)} = \sigma(z^{(1)})$$

$$z^{(2)} = w^T x^{(2)} + b$$

$$a^{(2)} = \sigma(z^{(2)})$$

$$z^{(3)} = w^T x^{(3)} + b$$

$$a^{(3)} = \sigma(z^{(3)})$$

We should do this m time

$$X = \begin{bmatrix} | & | & | & | \\ x^{(1)} & x^{(2)} & \dots & x^{(m)} \\ | & | & | & | \end{bmatrix} \quad (n_x, m)$$

$$w = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_{n_x} \end{bmatrix}$$

$$w^T = [w_1 \ w_2 \ \dots \ w_{n_x}]$$

$$Z = [z^{(1)} \ z^{(2)} \ \dots \ z^{(m)}] = w^T X + [b \ b \ \dots \ b] = [w^T x^{(1)} + b \quad w^T x^{(2)} + b \quad w^T x^{(3)} + b \quad \dots]$$

$$Z = np.dot(w.T, X) + b$$

b is (1,1)

*This is called "**Broadcasting**"*

$$A = [a^{(1)} \ a^{(2)} \ \dots \ a^{(m)}] = \sigma(Z)$$

Vectorization in back propagation

$$dz^{(1)} = a^{(1)} - y^{(1)} \quad dz^{(2)} = a^{(2)} - y^{(2)}$$

$$dZ = [dz^{(1)} \ dzz^{(2)} \ \dots \ dz^{(m)}] \quad (1 \times m)$$

$$A = [a^{(1)} a^{(2)} \ \dots \ a^{(m)}] \quad Y = [y^{(1)} y^{(2)} \ \dots \ y^{(m)}]$$

$$dz = A - Y = [a^{(1)} - y^{(1)} \ a^{(2)} - y^{(2)} \ \dots]$$

We got rid of for loop over number of features (nx). But the loop over training examples is still there

$$dw = 0$$

$$dw += X^{(1)} dz^{(1)}$$

$$dw += X^{(2)} dz^{(2)}$$

.

$$dw /= m$$

$$db = 0$$

$$db += dz^{(1)}$$

$$db += dz^{(2)}$$

.

$$db /= m$$

$$db = \frac{1}{m} \sum_{i=1}^m dz^{(i)}$$

$$= \frac{1}{m} \text{np.sum}(dz)$$

$$dw = \frac{1}{m} X dz^T$$

$$= \frac{1}{m} \begin{bmatrix} | & | & | & | \\ x^{(1)} & x^{(2)} & \dots & x^{(m)} \\ | & | & | & | \end{bmatrix} \begin{bmatrix} dz^1 \\ dz^2 \\ \cdot \\ \cdot \\ dz^m \end{bmatrix}$$

$$= \frac{1}{m} \begin{bmatrix} x_1^{(1)} dz^{(1)} + \dots + x_1^m dz^{(m)} \\ x_2^{(1)} dz^{(1)} + \dots + x_2^m dz^{(m)} \\ \cdot \\ \cdot \\ x_{n_x}^{(1)} dz^{(1)} + \dots + x_{n_x}^m dz^{(m)} \end{bmatrix} \quad (n_x \times 1)$$

Implementing Logistic Regression

$$J = 0, dw_1 = 0, dw_2 = 0, db = 0$$

for $i = 1$ to m

$$z^{(i)} = w^T X^{(i)} + b$$

$$a^{(i)} = \sigma(z^{(i)})$$

$$J += -[y^{(i)} \log a^{(i)} + (1 - y^{(i)}) \log(1 - a^{(i)})]$$

$$dz^{(i)} = a^{(i)} - y^{(i)}$$

$$dw_1 += x_1^{(i)} dz^{(i)}$$

$$dw_2 += x_2^{(i)} dz^{(i)}$$

$$db += dz^{(i)}$$

$$J /= m$$

$$dw_1 /= m$$

$$dw_2 /= m$$

$$db /= m$$

$$dw += X^{(i)} dz^{(i)}$$

For iteration in range (1000):

$$Z = w^T X + b$$

$$= np.dot(w.T, X) + b$$

$$A = \sigma(Z)$$

$$dZ = A - Y$$

$$dw = \frac{1}{m} X dZ^T = \frac{1}{m} (np.dot(X, dZ^T))$$

$$db = \frac{1}{m} np.sum(dZ)$$

$$w = w - \alpha \cdot dw$$

$$b = b - \alpha \cdot db$$

- It is a **single iteration** of gradient descent for logistic regression.
- For multiple iteration, we still need for loop.



What to do next

- Go to CANVAS
- Solve HW1



References

- Speech and Language Processing. Daniel Jurafsky & James H. Martin. Draft of January 7, 2023.
- [Deeplearning.ai](#)
- [Implementing-a-neural-network-from-scratch/](#)